

Product Design Document

5fing3rs — 3D Printing Cost Estimator

Team	Five Fingers Innovative Solutions
Project	5fing3rs 3D Printing Cost Estimator
Release	R1
Stack	FastAPI · PostgreSQL · React · TypeScript · Docker

This is a living document. It will grow with the implementation of the project and captures key design decisions made throughout development.

Contents

1	Design Model	2
1.1	Class Diagram	2
1.2	Class Descriptions	2
2	Sequence Diagrams	7
2.1	Use Case 1: User Registration with OTP Verification	8
2.2	Use Case 2: Upload STL File and Trigger Slicing	10
2.3	Use Case 3: Cost Estimation and Quote Download	11
2.4	Use Case 4: Admin Updates Material Pricing	13
3	Design Rationale	14
3.1	Decision 1: OTP Email Verification Before Login	14
3.2	Decision 2: Polling vs. WebSockets for Slicing Status	14
3.3	Decision 3: Custom Python/Trimesh Slicer vs. CuraEngine	15
3.4	Decision 4: Mailtrap API with Console Fallback	15
3.5	Decision 5: JWT in localStorage vs. HttpOnly Cookies	16
3.6	Decision 6: SQLite In-Memory Database for Test Suite	16

Class	State & Behaviour
Class 1 User (<i>app/models/user.py</i>)	Class state Maintains the identity and verification status of each registered user. • <code>id</code> — primary key • <code>username</code> — unique login name • <code>password_hash</code> — bcrypt hashed password • <code>email</code> — contact email (optional) • <code>role</code> — customer OR admin • <code>is_verified</code> — OTP verification status (bool) • <code>verification_code</code> — 6-digit OTP (nullable) • <code>verification_code_expires_at</code> — OTP expiry timestamp Class behaviour • Pure data model — no public methods. State is read and written exclusively by <code>AuthRouter</code>. • Maintains the verified/unverified state throughout the OTP registration flow.
Class 2 Job (<i>app/models/job.py</i>)	Class state Tracks the full lifecycle of a 3D print job from upload through slicing to cost estimation. • <code>job_id</code> — UUID primary key • <code>original_filename</code> — uploaded STL/STEP filename • <code>file_size</code> — size in bytes • <code>status</code> — lifecycle state: <code>uploaded</code> <code>slicing</code> <code>sliced</code> <code>failed</code> • <code>created_at</code> — creation timestamp Class behaviour • Pure data model — no public methods. • Status transitions are driven by <code>UploadRouter</code> (<code>uploaded</code>) and <code>SlicingRouter</code> (<code>slicing</code> → <code>sliced/failed</code>).

Class	State & Behaviour
Class 3 SlicingResult (<i>app/models/slicing_result.py</i>)	Class state Stores all numerical outputs from the slicing process for a given job. • <code>job_id</code> — FK to Job • <code>print_time_seconds</code> — estimated print duration • <code>material_weight_grams</code> — weight of material used • <code>support_material_grams</code> — weight of support material • <code>layer_count</code> — number of sliced layers • <code>slicing_parameters</code> — JSON of user-selected parameters Class behaviour • Pure data model produced by the trimesh slicer service. • Consumed by <code>CostCalculator</code> to produce the cost estimate.
Class 4 Material (<i>app/models/material.py</i>)	Class state Holds pricing and physical properties of each printable material. • <code>id</code> — primary key • <code>name</code> — material name (PLA, ABS, PETG, TPU) • <code>density_g_cm3</code> — material density in g/cm³ • <code>cost_per_gram</code> — price in INR per gram Class behaviour • Pure data model — managed via <code>AdminRouter</code> CRUD endpoints. • Read by <code>CostCalculator</code> to compute material cost.
Class 5 AdminConfig (<i>app/models/admin_config.py</i>)	Class state Key-value store for system-level configuration. • <code>key</code> — config key e.g. <code>machine_hourly_rate</code> • <code>value</code> — string value • <code>description</code> — human-readable description • <code>updated_at</code> — last modified timestamp Class behaviour • No public methods. • Provides machine rate, waste factor, and failure factor to <code>CostCalculator</code>.

Class	State & Behaviour
Class 6 AuthRouter (app/api/ auth.py)	Class state • SECRET_KEY, ALGORITHM — JWT signing configuration (from env) • ACCESS_TOKEN_EXPIRE_MINUTES — token lifetime Class behaviour • register(user_data, db) — creates User, sends OTP email • verify_otp(otp_data, db) — validates code, marks user verified, returns JWT • login(form_data, db) — validates credentials, returns JWT token • get_current_user(token, db) — decodes JWT, returns authenticated User • get_current_user_info() — returns current user's profile data
Class 7 CostCalculator (app/services/ cost_calculator.py)	Class state • Stateless service — no instance variables. Class behaviour • calculate_cost(db, slicing_result) — fetches Material and AdminConfig, computes material cost, support cost, machine time cost, applies waste and failure overhead factors, returns cost breakdown dict rounded to 2 decimal places (INR) • get_config_value(db, key, default) — retrieves a config value with fallback default

Class	State & Behaviour
Class 8 EmailService (app/services/ email_service.py)	Class state - MAILTRAP_API_TOKEN — loaded from environment - MAILTRAP_API_URL, MAILTRAP_FROM_EMAIL — email config Class behaviour - send_otp_email(user_email, code) — sends 6-digit OTP for verification - send_upload_success_email(job, user_email) — upload notification - send_slicing_complete_email(job, result, user_email) — slicing done notification - send_email(to, subject, html) — core send function; falls back to console log in dev mode if token not set
Class 9 AuthContext (frontend/src/ context/ AuthContext.tsx)	Class state - user — currently authenticated User object or null - loading — boolean while token is being validated on startup - isAuthenticated — derived: user !== null - isAdmin — derived: user.role === 'admin' Class behaviour - login(username, password) — calls POST /api/auth/login, stores JWT in localStorage - register(username, password, email) — calls POST /api/auth/register - verifyOtp(username, code) — calls POST /api/auth/verify-otp, stores JWT - logout() — clears localStorage, resets user state to null

2. Sequence Diagrams

UML sequence diagrams for the four major use cases of the 5fing3rs platform. Each diagram was generated using plantuml.com and is included as an image below.

2.1 Use Case 1: User Registration with OTP Verification

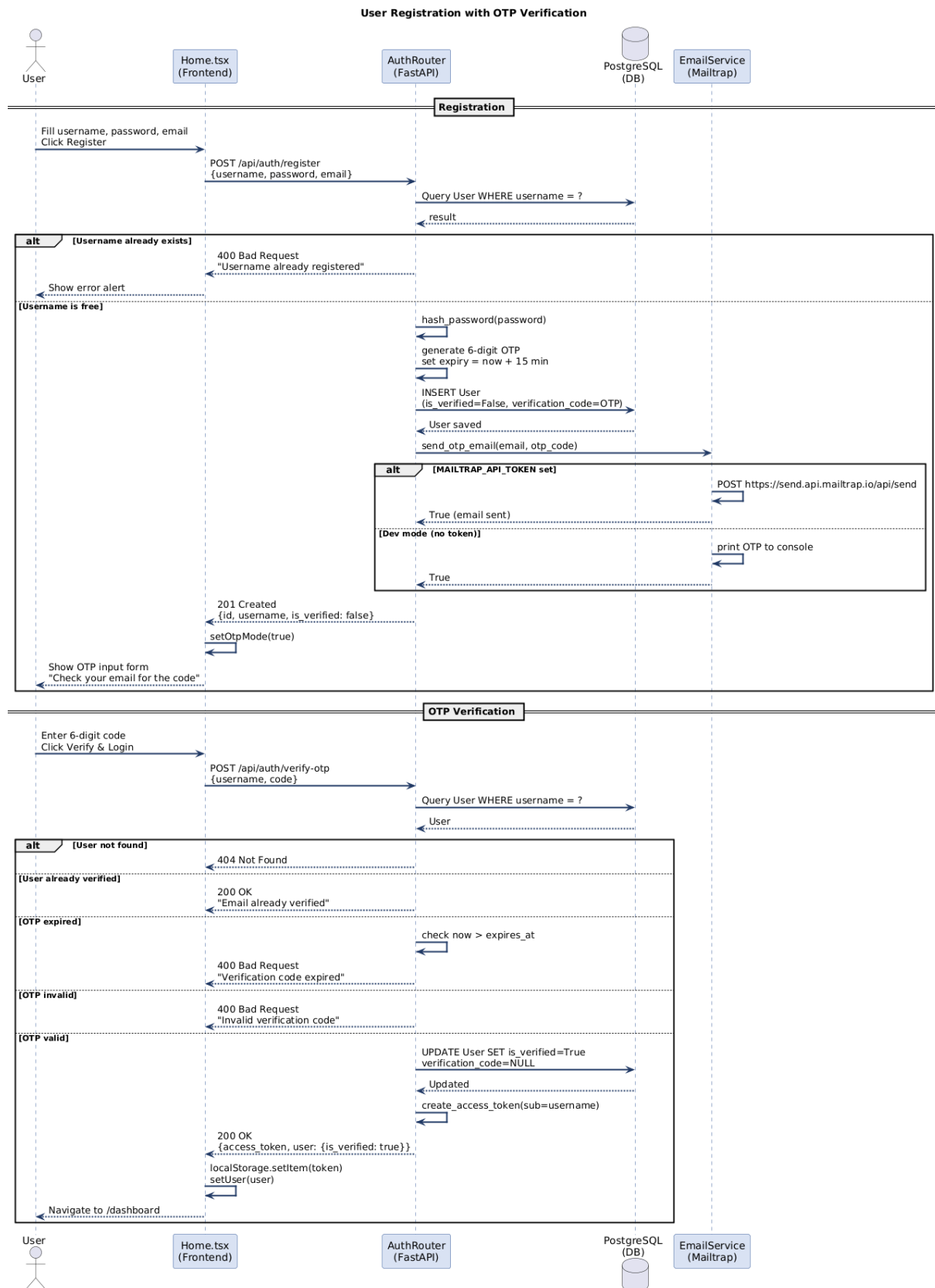
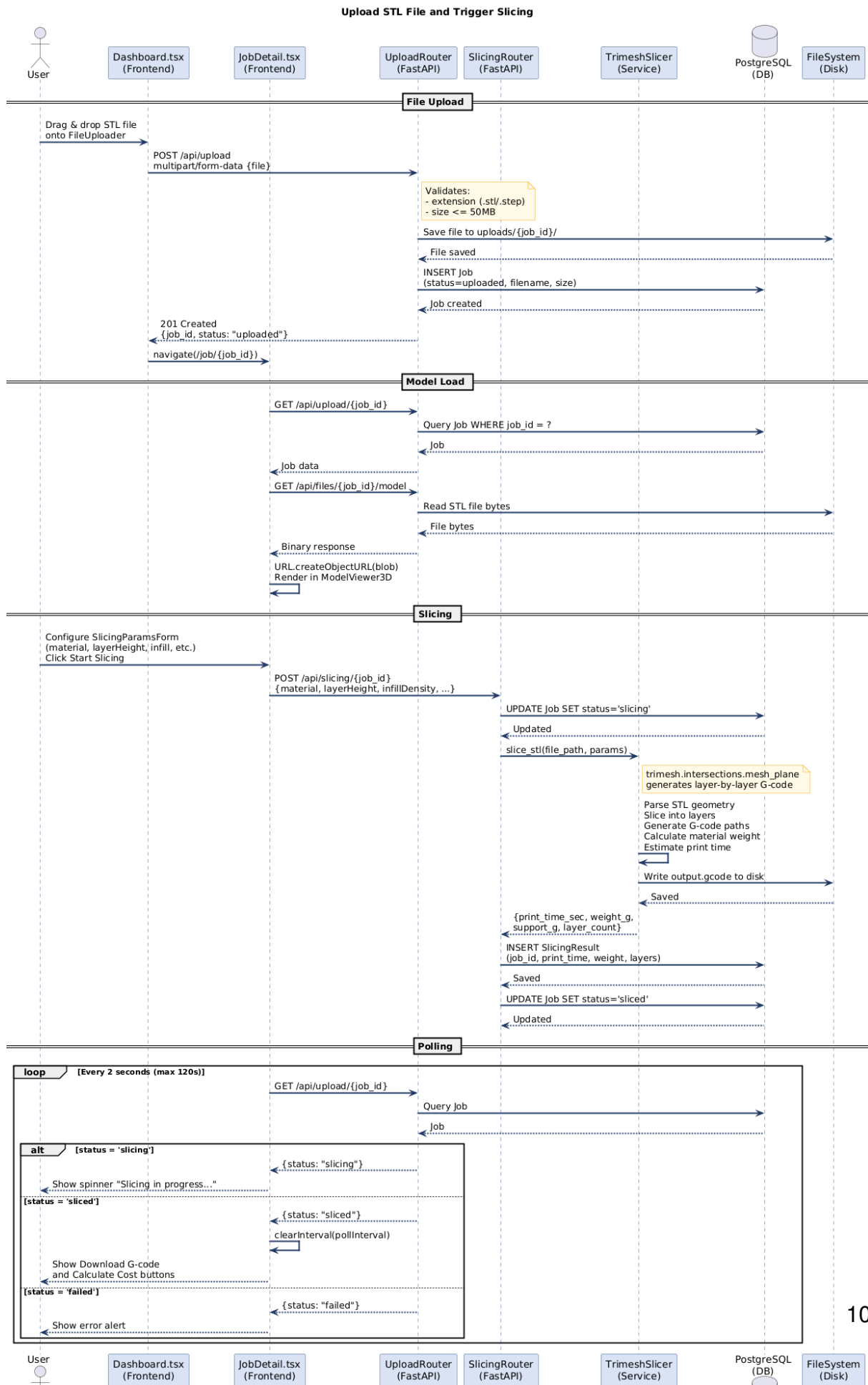


Figure 2: Sequence Diagram: User Registration with OTP Verification

2.2 Use Case 2: Upload STL File and Trigger Slicing



2.3 Use Case 3: Cost Estimation and Quote Download

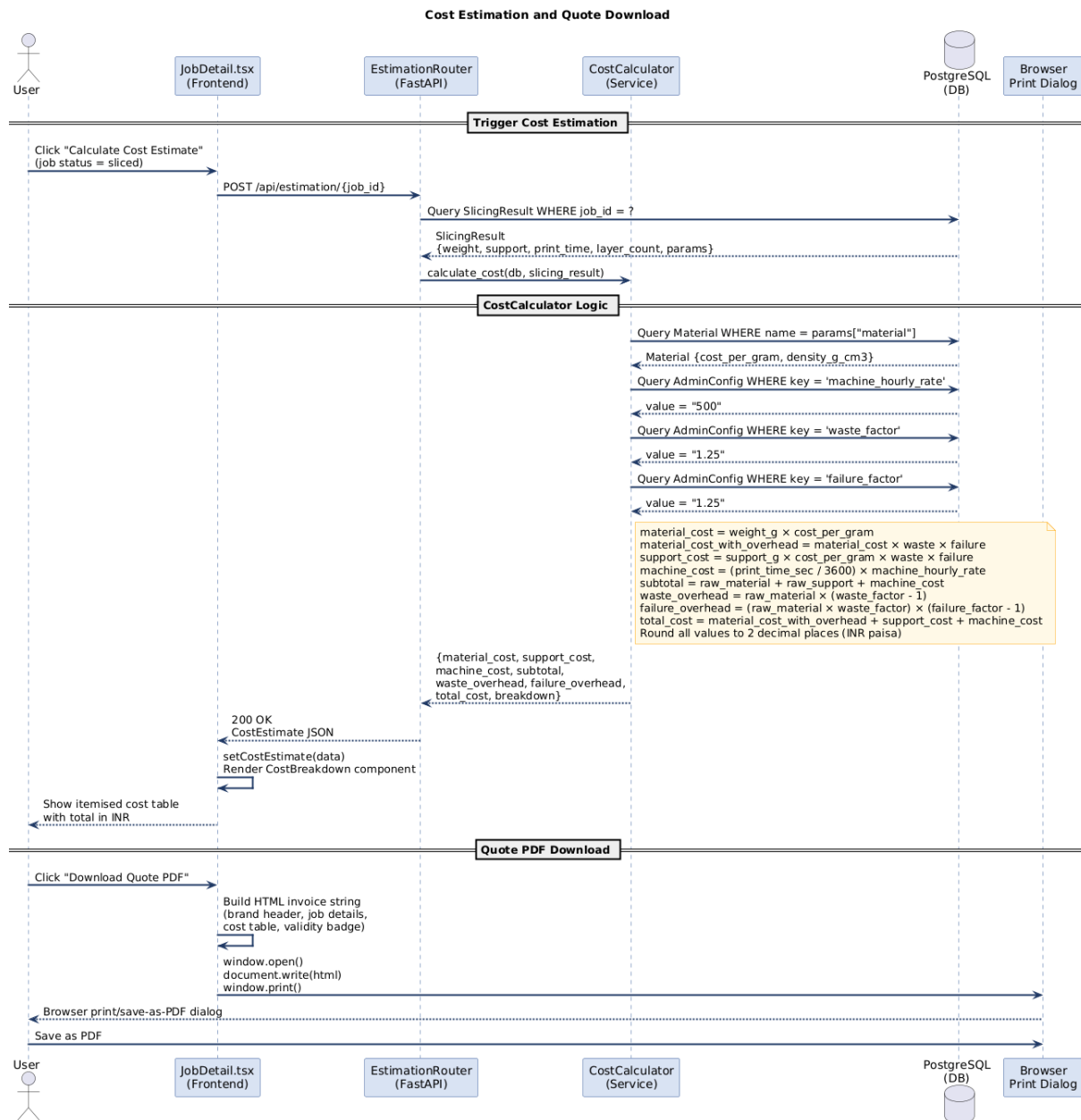


Figure 4: Sequence Diagram: Cost Estimation and Quote Download

2.4 Use Case 4: Admin Updates Material Pricing

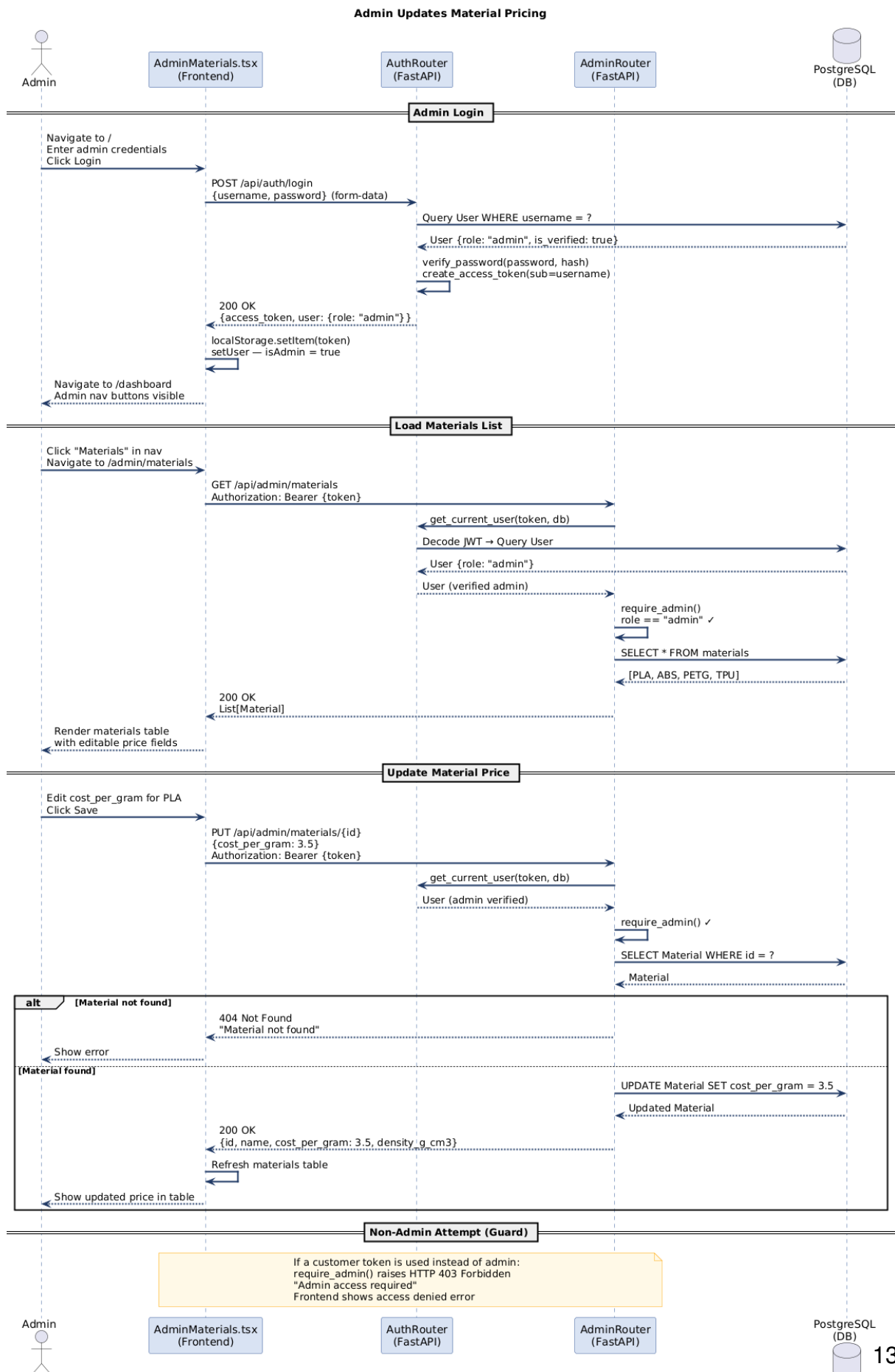


Figure 5: Sequence Diagram: Admin Updates Material Pricing

3. Design Rationale

This section records the key design decisions made during development of 5fing3rs. For each decision, alternatives that were considered and rejected are documented alongside the rationale for the final choice. This section will be updated whenever a significant design change occurs.

3.1 Decision 1: OTP Email Verification Before Login

Alternatives	Option A — Allow immediate login after registration; verify email lazily or make it optional. Option B — Require OTP verification before any authenticated action is permitted.
Decision	Option B — Users must verify their email via OTP before they can log in.
Rationale	Allowing unverified users to create jobs would pollute the database with records tied to fake or invalid emails. Since cost estimates and G-code files are generated and stored per user, data integrity requires confirmed email ownership. The tradeoff is a more complex registration flow, but this is standard practice for platforms that store user-generated files. A dev-mode fallback (OTP printed to the backend console when MAILTRAP_API_TOKEN is unset) ensures this does not block local development.

3.2 Decision 2: Polling vs. WebSockets for Slicing Status

Alternatives	Option A — WebSockets or Server-Sent Events (SSE) for real-time push when slicing completes. Option B — Client-side polling: frontend calls GET /api/upload/{job_id} every 2 seconds.
Decision	Option B — Polling every 2 seconds with a 2-minute timeout.
Rationale	WebSockets require persistent connection management on the backend, additional CORS and proxy configuration, and more complex frontend state handling. Since slicing jobs typically complete within 30–120 seconds and the polling interval is short (2 seconds), the user experience difference is negligible. Polling is stateless, simpler to implement, and requires no changes to the FastAPI server configuration. If job durations grow significantly in future releases, SSE can be introduced as a targeted upgrade.

3.3 Decision 3: Custom Python/Trimesh Slicer vs. CuraEngine

Alternatives	Option A — CuraEngine, the industry-standard C++ slicer used by Ultimaker Cura. Option B — Custom slicer built with Python and the <code>trimesh</code> library.
Decision	Option B — Custom trimesh-based Python slicer.
Rationale	CuraEngine is a large C++ binary with complex build dependencies that are difficult to bundle cross-platform inside a Docker container, particularly on ARM architectures. The custom trimesh slicer installs via pip alongside other Python dependencies, making it fully portable and Docker-compatible with no additional build steps. The tradeoff is lower G-code quality compared to Cura (no advanced optimisations like combing or seam placement), but for the purpose of cost estimation the material weight and print time outputs are sufficiently accurate for pricing.

3.4 Decision 4: Mailtrap API with Console Fallback

Alternatives	Option A — Require a real SMTP server for all environments; fail hard if credentials are absent. Option B — Use Mailtrap API with a graceful fallback that prints OTP to the backend console.
Decision	Option B — Mailtrap with dev-mode console fallback.
Rationale	Requiring real SMTP credentials blocks developers from running the project locally without an email provider account. The console fallback allows the full OTP verification flow to be tested locally — the developer copies the OTP from the terminal. In production, setting <code>MAILTRAP_API_TOKEN</code> in <code>.env</code> switches to real email delivery transparently. This also avoids accidentally sending test emails to real addresses during development.

3.5 Decision 5: JWT in localStorage vs. HttpOnly Cookies

Alternatives	Option A — Store JWT in an <code>httpOnly</code> cookie set by the server (XSS-resistant). Option B — Store JWT in <code>localStorage</code> , accessible to JavaScript for Authorization headers.
Decision	Option B — JWT stored in <code>localStorage</code> .
Rationale	HttpOnly cookies require server-side cookie management, CSRF protection, and SameSite configuration, adding complexity to both FastAPI and React. Since 5fing3rs is an internal platform for a known set of users (not a public consumer app), the XSS risk profile is lower. <code>localStorage</code> is simpler to implement with Axios interceptors and is the standard approach for React SPAs with token-based auth. Migrating to <code>httpOnly</code> cookies is recommended if the platform is deployed publicly in a future release.

3.6 Decision 6: SQLite In-Memory Database for Test Suite

Alternatives	Option A — Spin up a real PostgreSQL container in CI using GitHub Actions services. Option B — Override the DB dependency with SQLite in-memory via <code>conftest.py</code> .
Decision	Option B — SQLite in-memory database for all tests.
Rationale	Running a PostgreSQL service in CI adds startup time, increases pipeline complexity, and requires managing container health checks. SQLite in-memory databases start instantly, require zero configuration, and are destroyed after each test giving complete isolation. The FastAPI dependency injection system allows the database session to be cleanly overridden per test via <code>app.dependency_overrides</code> . The tradeoff is that PostgreSQL-specific features must be handled using SQLAlchemy's database-agnostic types, which was done throughout the model definitions.